

# Data Storage

## Lecture 2

# Data Storage

- Bits and Their Storage
- Main Memory
- Mass Storage
- Representing Information as Bit Patterns
- Binary System

# Bits and Bit Patterns

- **Bit:** Binary Digit (0 or 1)
- Bit Patterns are used to represent information
  - Numbers
  - Text characters
  - Images
  - Sound
  - And others
- Non-binary computers made of multi-level electrical components are possible, but not popular yet



Claude Shannon  
(1916 - 2001)

# Boolean Operations

- Computers are made of small devices that can compute Boolean functions extremely fast
- **Boolean Operation:** An operation that manipulates one or more true/false values
- Specific operations
  - AND ( $\wedge$ )
  - OR ( $\vee$ )
  - XOR ( $\oplus$ ) - exclusive OR
  - NOT ( $\neg$ )



George Boole

1815 - 1864

# The possible input and output values of Boolean operations AND, OR, and XOR (exclusive or)

## The AND operation

$$\begin{array}{r} 0 \\ \text{AND } 0 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 0 \\ \text{AND } 1 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 1 \\ \text{AND } 0 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 1 \\ \text{AND } 1 \\ \hline 1 \end{array}$$

## The OR operation

$$\begin{array}{r} 0 \\ \text{OR } 0 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 0 \\ \text{OR } 1 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 1 \\ \text{OR } 0 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 1 \\ \text{OR } 1 \\ \hline 1 \end{array}$$

## The XOR operation

$$\begin{array}{r} 0 \\ \text{XOR } 0 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 0 \\ \text{XOR } 1 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 1 \\ \text{XOR } 0 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 1 \\ \text{XOR } 1 \\ \hline 0 \end{array}$$

# Properties of Conjunction (AND) and Disjunction (OR)

- **Commutativity:**

- $A \text{ AND } B = B \text{ AND } A$
- $A \text{ OR } B = B \text{ OR } A$

- **Associativity:**

- $A \text{ AND } (B \text{ AND } C) = (A \text{ AND } B) \text{ AND } C$
- $A \text{ OR } (B \text{ OR } C) = (A \text{ OR } B) \text{ OR } C$

- **Distributivity:**

- $A \text{ AND } (B \text{ OR } C) = (A \text{ AND } B) \text{ OR } (A \text{ AND } C)$
- $A \text{ OR } (B \text{ AND } C) = (A \text{ OR } B) \text{ AND } (A \text{ OR } C)$

- **Idempotency:**

- $A = A \text{ AND } A$
- $A = A \text{ OR } A$

# De Morgan's Laws

- $\text{NOT}(A) \text{ OR } A = 1$
- $\text{NOT}(A) \text{ AND } A = 0$
- $\text{NOT}(\text{NOT}(A)) = A$
- $\text{NOT}(A \text{ AND } B) = \text{NOT}(A) \text{ OR } \text{NOT}(B)$
- $\text{NOT}(A \text{ OR } B) = \text{NOT}(A) \text{ AND } \text{NOT}(B)$
- Exercise:
  - What is  $\text{NOT}(\text{NOT}(A \text{ AND } B) \text{ AND } \text{NOT}(A \text{ AND } B))$ ?

# Boolean algebra

Prove

$$\text{Not}(x \text{ And } y) = \text{Not}(x) \text{ Or } \text{Not}(y)$$

$f(x,y) = \text{Not}(x \text{ And } y)$

| $x$ | $y$ | $f$ |
|-----|-----|-----|
| 0   | 0   |     |
| 0   | 1   |     |
| 1   | 0   |     |
| 1   | 1   |     |

$g(x,y) = \text{Not}(x) \text{ Or } \text{Not}(y)$

| $x$ | $y$ | $g$ |
|-----|-----|-----|
| 0   | 0   |     |
| 0   | 1   |     |
| 1   | 0   |     |
| 1   | 1   |     |

Proof: Fill in the right column in both truth tables.

If  $f = g$ , the identity is proved.

# Boolean algebra

Prove

$$\text{Not}(x \text{ And } y) = \text{Not}(x) \text{ Or } \text{Not}(y)$$

| $f(x,y) = \text{Not}(x \text{ And } y)$ |     |     | $g(x,y) = \text{Not}(x) \text{ Or } \text{Not}(y)$ |     |     |
|-----------------------------------------|-----|-----|----------------------------------------------------|-----|-----|
| $x$                                     | $y$ | $f$ | $x$                                                | $y$ | $g$ |
| 0                                       | 0   | 1   | 0                                                  | 0   | 1   |
| 0                                       | 1   | 1   | 0                                                  | 1   | 1   |
| 1                                       | 0   | 1   | 1                                                  | 0   | 1   |
| 1                                       | 1   | 0   | 1                                                  | 1   | 0   |

Proof: Fill in the right column in both truth tables.

If  $f = g$ , the identity is proved.

# Boolean algebra

Commutative:  $x \text{ And } y = y \text{ And } x$   
 $x \text{ Or } y = y \text{ Or } x$

Associative:  $x \text{ And } (y \text{ And } z) = (x \text{ And } y) \text{ And } z$   
 $x \text{ Or } (y \text{ Or } z) = (x \text{ Or } y) \text{ Or } z$

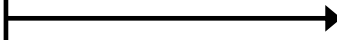
Distributive:  $x \text{ And } (y \text{ Or } z) = (x \text{ And } y) \text{ Or } (x \text{ And } z)$   
 $x \text{ Or } (y \text{ And } z) = (x \text{ Or } y) \text{ And } (x \text{ Or } z)$

De Morgan:  $\text{Not}(x \text{ And } y) = \text{Not}(x) \text{ Or } \text{Not}(y)$   
 $\text{Not}(x \text{ Or } y) = \text{Not}(x) \text{ And } \text{Not}(y)$

Idempotent:  $x \text{ And } x = x$   
 $x \text{ Or } x = x$

Double negation:

$$\text{Not}(\text{Not}(x)) = x$$



## Substitution:

In any such identity,  $x$  and  $y$  can be substituted with any boolean function

## Substitution examples:

$$\text{Not}(\text{Not}(a)) = a$$

$$\text{Not}(\text{Not}(u \text{ Or } v)) = u \text{ Or } v$$

Etc.

# Boolean algebra

Commutative:  $x \text{ And } y = y \text{ And } x$   
 $x \text{ Or } y = y \text{ Or } x$

Associative:  $x \text{ And } (y \text{ And } z) = (x \text{ And } y) \text{ And } z$   
 $x \text{ Or } (y \text{ Or } z) = (x \text{ Or } y) \text{ Or } z$

Distributive:  $x \text{ And } (y \text{ Or } z) = (x \text{ And } y) \text{ Or } (x \text{ And } z)$   
 $x \text{ Or } (y \text{ And } z) = (x \text{ Or } y) \text{ And } (x \text{ Or } z)$

De Morgan:  $\text{Not}(x \text{ And } y) = \text{Not}(x) \text{ Or } \text{Not}(y)$   
 $\text{Not}(x \text{ Or } y) = \text{Not}(x) \text{ And } \text{Not}(y)$

Idempotent:  $x \text{ And } x = x$   
 $x \text{ Or } x = x$

Double negation:  $\text{Not}(\text{Not}(x)) = x$

Task: Simplify this function (example):

**$\text{Not}(\text{Not}(x) \text{ And } \text{Not}(x \text{ Or } y))$**



# Boolean algebra

Commutative:  $x \text{ And } y = y \text{ And } x$   
 $x \text{ Or } y = y \text{ Or } x$

Associative:  $x \text{ And } (y \text{ And } z) = (x \text{ And } y) \text{ And } z$   
 $x \text{ Or } (y \text{ Or } z) = (x \text{ Or } y) \text{ Or } z$

Distributive:  $x \text{ And } (y \text{ Or } z) = (x \text{ And } y) \text{ Or } (x \text{ And } z)$   
 $x \text{ Or } (y \text{ And } z) = (x \text{ Or } y) \text{ And } (x \text{ Or } z)$

De Morgan:  $\text{Not}(x \text{ And } y) = \text{Not}(x) \text{ Or } \text{Not}(y)$   
 $\text{Not}(x \text{ Or } y) = \text{Not}(x) \text{ And } \text{Not}(y)$

Idempotent:  $x \text{ And } x = x$   
 $x \text{ Or } x = x$

Double negation:  $\text{Not}(\text{Not}(x)) = x$

Task: Simplify this function (example):

**$\text{Not}(\text{Not}(x) \text{ And } \text{Not}(x \text{ Or } y)) =$**

By De Morgan's rule:

**$\text{Not}(\text{Not}(x) \text{ And } (\text{Not}(x) \text{ And } \text{Not}(y)))$**



# Boolean algebra

Commutative:  $x \text{ And } y = y \text{ And } x$   
 $x \text{ Or } y = y \text{ Or } x$

Associative:  $x \text{ And } (y \text{ And } z) = (x \text{ And } y) \text{ And } z$   
 $x \text{ Or } (y \text{ Or } z) = (x \text{ Or } y) \text{ Or } z$

Distributive:  $x \text{ And } (y \text{ Or } z) = (x \text{ And } y) \text{ Or } (x \text{ And } z)$   
 $x \text{ Or } (y \text{ And } z) = (x \text{ Or } y) \text{ And } (x \text{ Or } z)$

De Morgan:  $\text{Not}(x \text{ And } y) = \text{Not}(x) \text{ Or } \text{Not}(y)$   
 $\text{Not}(x \text{ Or } y) = \text{Not}(x) \text{ And } \text{Not}(y)$

Idempotent:  $x \text{ And } x = x$   
 $x \text{ Or } x = x$

Double negation:  $\text{Not}(\text{Not}(x)) = x$

Task: Simplify this function (example):

**$\text{Not}(\text{Not}(x) \text{ And } \text{Not}(x \text{ Or } y)) =$**

By De Morgan's rule:

$\text{Not}(\text{Not}(x) \text{ And } (\text{Not}(x) \text{ And } \text{Not}(y))) =$

By the associative  
rule:

$\text{Not}((\text{Not}(x) \text{ And } \text{Not}(x)) \text{ And } \text{Not}(y)) =$



# Boolean algebra

Commutative:  $x \text{ And } y = y \text{ And } x$   
 $x \text{ Or } y = y \text{ Or } x$

Associative:  $x \text{ And } (y \text{ And } z) = (x \text{ And } y) \text{ And } z$   
 $x \text{ Or } (y \text{ Or } z) = (x \text{ Or } y) \text{ Or } z$

Distributive:  $x \text{ And } (y \text{ Or } z) = (x \text{ And } y) \text{ Or } (x \text{ And } z)$   
 $x \text{ Or } (y \text{ And } z) = (x \text{ Or } y) \text{ And } (x \text{ Or } z)$

De Morgan:  $\text{Not}(x \text{ And } y) = \text{Not}(x) \text{ Or } \text{Not}(y)$   
 $\text{Not}(x \text{ Or } y) = \text{Not}(x) \text{ And } \text{Not}(y)$

Idempotent:  $x \text{ And } x = x$   
 $x \text{ Or } x = x$

Double negation:  $\text{Not}(\text{Not}(x)) = x$

Task: Simplify this function (example):

**$\text{Not}(\text{Not}(x) \text{ And } \text{Not}(x \text{ Or } y)) =$**

By De Morgan's rule:

$\text{Not}(\text{Not}(x) \text{ And } (\text{Not}(x) \text{ And } \text{Not}(y))) =$

By the associative rule:

$\text{Not}((\text{Not}(x) \text{ And } \text{Not}(x)) \text{ And } \text{Not}(y)) =$

By the idempotent rule:

$\text{Not}(\text{Not}(x) \text{ And } \text{Not}(y))$



# Boolean algebra

Commutative:  $x \text{ And } y = y \text{ And } x$   
 $x \text{ Or } y = y \text{ Or } x$

Associative:  $x \text{ And } (y \text{ And } z) = (x \text{ And } y) \text{ And } z$   
 $x \text{ Or } (y \text{ Or } z) = (x \text{ Or } y) \text{ Or } z$

Distributive:  $x \text{ And } (y \text{ Or } z) = (x \text{ And } y) \text{ Or } (x \text{ And } z)$   
 $x \text{ Or } (y \text{ And } z) = (x \text{ Or } y) \text{ And } (x \text{ Or } z)$

De Morgan:  $\text{Not}(x \text{ And } y) = \text{Not}(x) \text{ Or } \text{Not}(y)$   
 $\text{Not}(x \text{ Or } y) = \text{Not}(x) \text{ And } \text{Not}(y)$

Idempotent:  $x \text{ And } x = x$   
 $x \text{ Or } x = x$

Double negation:  $\text{Not}(\text{Not}(x)) = x$

Task: Simplify this function (example):

**$\text{Not}(\text{Not}(x) \text{ And } \text{Not}(x \text{ Or } y)) =$**

By De Morgan's rule:

$\text{Not}(\text{Not}(x) \text{ And } (\text{Not}(x) \text{ And } \text{Not}(y))) =$

By the associative rule:

$\text{Not}((\text{Not}(x) \text{ And } \text{Not}(x)) \text{ And } \text{Not}(y)) =$

By the idempotent rule:

$\text{Not}(\text{Not}(x) \text{ And } \text{Not}(y)) =$

By De Morgan's rule:

$\text{Not}(\text{Not}(x \text{ Or } y))$



# Boolean algebra

Commutative:  $x \text{ And } y = y \text{ And } x$   
 $x \text{ Or } y = y \text{ Or } x$

Associative:  $x \text{ And } (y \text{ And } z) = (x \text{ And } y) \text{ And } z$   
 $x \text{ Or } (y \text{ Or } z) = (x \text{ Or } y) \text{ Or } z$

Distributive:  $x \text{ And } (y \text{ Or } z) = (x \text{ And } y) \text{ Or } (x \text{ And } z)$   
 $x \text{ Or } (y \text{ And } z) = (x \text{ Or } y) \text{ And } (x \text{ Or } z)$

De Morgan:  $\text{Not}(x \text{ And } y) = \text{Not}(x) \text{ Or } \text{Not}(y)$   
 $\text{Not}(x \text{ Or } y) = \text{Not}(x) \text{ And } \text{Not}(y)$

Idempotent:  $x \text{ And } x = x$   
 $x \text{ Or } x = x$

Double negation:  $\text{Not}(\text{Not}(x)) = x$

Task: Simplify this function (example):

**$\text{Not}(\text{Not}(x) \text{ And } \text{Not}(x \text{ Or } y)) =$**

By De Morgan's rule:

$\text{Not}(\text{Not}(x) \text{ And } (\text{Not}(x) \text{ And } \text{Not}(y))) =$

By the associative rule:

$\text{Not}((\text{Not}(x) \text{ And } \text{Not}(x)) \text{ And } \text{Not}(y)) =$

By the idempotent rule:

$\text{Not}(\text{Not}(x) \text{ And } \text{Not}(y)) =$

By De Morgan's rule:

$\text{Not}(\text{Not}(x \text{ Or } y)) =$

By double negation:

**$x \text{ Or } y$**

# Boolean algebra

## Observations about simplifying Boolean functions:

- Can lead to significant optimization
- Based on intuition, experience, and luck
- Can be assisted by some tools
- But, in general: *NP*-hard.

Task: Simplify this function (example):

$$\text{Not}(\text{Not}(x) \text{ And } \text{Not}(x \text{ Or } y)) =$$

By De Morgan's rule:

$$\text{Not}(\text{Not}(x) \text{ And } (\text{Not}(x) \text{ And } \text{Not}(y))) =$$

By the associative rule:

$$\text{Not}((\text{Not}(x) \text{ And } \text{Not}(x)) \text{ And } \text{Not}(y)) =$$

By the idempotent rule:

$$\text{Not}(\text{Not}(x) \text{ And } \text{Not}(y)) =$$

By De Morgan's rule:

$$\text{Not}(\text{Not}(x \text{ Or } y)) =$$

By double negation:

$$x \text{ Or } y$$

# Boolean algebra

Commutative:  $(x \text{ And } y) = (y \text{ And } x)$   
 $(x \text{ Or } y) = (y \text{ Or } x)$

Associative:  $(x \text{ And } (y \text{ And } z)) = ((x \text{ And } y) \text{ And } z)$   
 $(x \text{ Or } (y \text{ Or } z)) = ((x \text{ Or } y) \text{ Or } z)$

Distributive:  $(x \text{ And } (y \text{ Or } z)) = (x \text{ And } y) \text{ Or } (x \text{ And } z)$   
 $(x \text{ Or } (y \text{ And } z)) = (x \text{ Or } y) \text{ And } (x \text{ Or } z)$

De Morgan:  $\text{Not}(x \text{ And } y) = \text{Not}(x) \text{ Or } \text{Not}(y)$   
 $\text{Not}(x \text{ Or } y) = \text{Not}(x) \text{ And } \text{Not}(y)$

Idempotent:  $(x \text{ And } x) = x$   
 $(x \text{ Or } x) = x$

Double negation:  $\text{Not}(\text{Not}(x)) = x$

Another example: Prove that

$$x \text{ Or } y = \text{Not}(\text{Not}(x) \text{ And } \text{Not}(y))$$

# Boolean algebra

Commutative:  $(x \text{ And } y) = (y \text{ And } x)$   
 $(x \text{ Or } y) = (y \text{ Or } x)$

Associative:  $(x \text{ And } (y \text{ And } z)) = ((x \text{ And } y) \text{ And } z)$   
 $(x \text{ Or } (y \text{ Or } z)) = ((x \text{ Or } y) \text{ Or } z)$

Distributive:  $(x \text{ And } (y \text{ Or } z)) = (x \text{ And } y) \text{ Or } (x \text{ And } z)$   
 $(x \text{ Or } (y \text{ And } z)) = (x \text{ Or } y) \text{ And } (x \text{ Or } z)$

De Morgan:  $\text{Not}(x \text{ And } y) = \text{Not}(x) \text{ Or } \text{Not}(y)$   
 $\text{Not}(x \text{ Or } y) = \text{Not}(x) \text{ And } \text{Not}(y)$

Idempotent:  $(x \text{ And } x) = x$   
 $(x \text{ Or } x) = x$

Double negation:  $\text{Not}(\text{Not}(x)) = x$

Another example: Prove that

$$x \text{ Or } y = \text{Not}(\text{Not}(x) \text{ And } \text{Not}(y))$$

De Morgan:

$$\text{Not}(x \text{ Or } y) = \text{Not}(x) \text{ And } \text{Not}(y)$$

Negate both sides:

$$\text{Not}(\text{Not}(x \text{ Or } y)) = \text{Not}(\text{Not}(x) \text{ And } \text{Not}(y))$$

By double negation:

$$x \text{ Or } y = \text{Not}(\text{Not}(x) \text{ And } \text{Not}(y))$$

## Implication

- We don't really "need" Or
- We will soon revisit this reduction

# Boolean function

Formula (example)

$$f(x, y, z) = (x \text{ And } (\text{Not}(y) \text{ Or } z)) \text{ And } y$$

Truth table

| $x$ | $y$ | $z$ | $f$ |
|-----|-----|-----|-----|
| 0   | 0   | 0   | 0   |
| 0   | 0   | 1   | 1   |
| 0   | 1   | 0   | 0   |
| 0   | 1   | 1   | 1   |
| 1   | 0   | 0   | 0   |
| 1   | 0   | 1   | 0   |
| 1   | 1   | 0   | 1   |
| 1   | 1   | 1   | 1   |

A Boolean function can be expressed using a *formula*, or a *truth table*

The two representations are equivalent

Question: Can we construct each representation from the other one?

Formula ➡ truth table

Formula

$$f(x, y, z) = (x \text{ And } (\text{Not}(y) \text{ Or } z)) \text{ And } y$$

Formula ➡ truth table

Formula

$$f(x, y, z) = (x \text{ And } (\text{Not}(y) \text{ Or } z)) \text{ And } y$$



Truth table

| <i>x</i> | <i>y</i> | <i>z</i> | <i>f</i> |
|----------|----------|----------|----------|
| 0        | 0        | 0        |          |
| 0        | 0        | 1        |          |
| 0        | 1        | 0        |          |
| 0        | 1        | 1        |          |
| 1        | 0        | 0        |          |
| 1        | 0        | 1        |          |
| 1        | 1        | 0        |          |
| 1        | 1        | 1        |          |

Formula ➡ truth table

Formula

$$f(x, y, z) = (x \text{ And } (\text{Not}(y) \text{ Or } z)) \text{ And } y$$



Truth table

| $x$ | $y$ | $z$ | $f$ |
|-----|-----|-----|-----|
| 0   | 0   | 0   | 0   |
| 0   | 0   | 1   | 0   |
| 0   | 1   | 0   |     |
| 0   | 1   | 1   |     |
| 1   | 0   | 0   | 0   |
| 1   | 0   | 1   | 0   |
| 1   | 1   | 0   |     |
| 1   | 1   | 1   |     |

When  $y = 0$ ,  $f$  must be  
0

Formula ➡ truth table

Formula

$$f(x, y, z) = (x \text{ And } (\text{Not}(y) \text{ Or } z)) \text{ And } y$$



Truth table

| $x$ | $y$ | $z$ | $f$ |
|-----|-----|-----|-----|
| 0   | 0   | 0   | 0   |
| 0   | 0   | 1   | 0   |
| 0   | 1   | 0   | 0   |
| 0   | 1   | 1   | 0   |
| 1   | 0   | 0   | 0   |
| 1   | 0   | 1   | 0   |
| 1   | 1   | 0   |     |
| 1   | 1   | 1   |     |

When  $y = 0$ ,  $f$  must be  
0

When  $x = 0$ ,  $f$  must be  
0

Formula ➡ truth table

Formula

$$f(x, y, z) = (x \text{ And } (\text{Not}(y) \text{ Or } z)) \text{ And } y$$



Truth table

| $x$ | $y$ | $z$ | $f$ |
|-----|-----|-----|-----|
| 0   | 0   | 0   | 0   |
| 0   | 0   | 1   | 0   |
| 0   | 1   | 0   | 0   |
| 0   | 1   | 1   | 0   |
| 1   | 0   | 0   | 0   |
| 1   | 0   | 1   | 0   |
| 1   | 1   | 0   | 1   |
| 1   | 1   | 1   | 1   |

When  $y = 0$ ,  $f$  must be  
0

When  $x = 0$ ,  $f$  must be  
0

$(1 \text{ And } (\text{Not}(1) \text{ Or } 0)) \text{ And } 1 =$   
1

$(1 \text{ And } (\text{Not}(1) \text{ Or } 1)) \text{ And } 1 =$   
1

Formula ➡ truth table

Formula

$$f(x, y, z) = (x \text{ And } (\text{Not}(y) \text{ Or } z)) \text{ And } y$$



Truth table

| $x$ | $y$ | $z$ | $f$ |
|-----|-----|-----|-----|
| 0   | 0   | 0   | 0   |
| 0   | 0   | 1   | 0   |
| 0   | 1   | 0   | 0   |
| 0   | 1   | 1   | 0   |
| 1   | 0   | 0   | 0   |
| 1   | 0   | 1   | 0   |
| 1   | 1   | 0   | 1   |
| 1   | 1   | 1   | 1   |

Lemma

Given a Boolean function expressed as a formula, we can always construct from it its truth table.

Proof: Evaluate the function over all the possible values of its variables (which is *the* definition of a truth table)

## Boolean function synthesis: Truth table → formula

### Formula

$$f(x, y, z) = (x \text{ And } (\text{Not}(y) \text{ Or } z)) \text{ And } y$$

### Truth table



| $x$ | $y$ | $z$ | $f$ |
|-----|-----|-----|-----|
| 0   | 0   | 0   | 0   |
| 0   | 0   | 1   | 0   |
| 0   | 1   | 0   | 0   |
| 0   | 1   | 1   | 0   |
| 1   | 0   | 0   | 0   |
| 1   | 0   | 1   | 0   |
| 1   | 1   | 0   | 1   |
| 1   | 1   | 1   | 1   |

Can we also go the other way around,  
*for any given truth table?*

## Boolean function synthesis: Truth table $\Rightarrow$ formula

| $x$ | $y$ | $z$ | $f$ |
|-----|-----|-----|-----|
| 0   | 0   | 0   | 1   |
| 0   | 0   | 1   | 0   |
| 0   | 1   | 0   | 1   |
| 0   | 1   | 1   | 0   |
| 1   | 0   | 0   | 1   |
| 1   | 0   | 1   | 0   |
| 1   | 1   | 0   | 0   |
| 1   | 1   | 1   | 0   |

Goal: Synthesize a formula which is equivalent to this truth table

# Boolean function synthesis



| $x$ | $y$ | $z$ | $f$ |
|-----|-----|-----|-----|
| 0   | 0   | 0   | 1   |
| 0   | 0   | 1   | 0   |
| 0   | 1   | 0   | 1   |
| 0   | 1   | 1   | 0   |
| 1   | 0   | 0   | 1   |
| 1   | 0   | 1   | 0   |
| 1   | 1   | 0   | 0   |
| 1   | 1   | 1   | 0   |

1. Focus on the rows where  $f = 1$

# Boolean function synthesis

| $x$ | $y$ | $z$ | $f$ | $f_1$ |
|-----|-----|-----|-----|-------|
| 0   | 0   | 0   | 1   | 1     |
| 0   | 0   | 1   | 0   | 0     |
| 0   | 1   | 0   | 1   | 0     |
| 0   | 1   | 1   | 0   | 0     |
| 1   | 0   | 0   | 1   | 0     |
| 1   | 0   | 1   | 0   | 0     |
| 1   | 1   | 0   | 0   | 0     |
| 1   | 1   | 1   | 0   | 0     |

Not(x) And Not(y) And  
Not(z)

1. Focus on the rows where  $f = 1$
2. For each such row  $i$ , define a function  $f_i$  which equals 1 in row  $i$  and 0 elsewhere.  
Define  $f_i$  to be a conjunction of all of  $f$ 's variables or their negations, as the variable is 1 or 0 in the  $i$ 'th row

# Boolean function synthesis

|   | $x$ | $y$ | $z$ | $f$ | $f_1$ | $f_3$ |
|---|-----|-----|-----|-----|-------|-------|
| → | 0   | 0   | 0   | 1   | 1     | 0     |
|   | 0   | 0   | 1   | 0   | 0     | 0     |
| → | 0   | 1   | 0   | 1   | 0     | 1     |
|   | 0   | 1   | 1   | 0   | 0     | 0     |
| → | 1   | 0   | 0   | 1   | 0     | 0     |
|   | 1   | 0   | 1   | 0   | 0     | 0     |
|   | 1   | 1   | 0   | 0   | 0     | 0     |
|   | 1   | 1   | 1   | 0   | 0     | 0     |

Not(x) And Not(y) And  
Not(z)

Not(x) And y And Not(z)

1. Focus on the rows where  $f = 1$
2. For each such row  $i$ , define a function  $f_i$  which equals 1 in row  $i$  and 0 elsewhere.  
Define  $f_i$  to be a conjunction of all of  $f$ 's variables or their negations, as the variable is 1 or 0 in the  $i$ 'th row

# Boolean function synthesis

|   | $x$ | $y$ | $z$ | $f$ | $f_1$ | $f_3$ | $f_5$ |                                                                       |
|---|-----|-----|-----|-----|-------|-------|-------|-----------------------------------------------------------------------|
| → | 0   | 0   | 0   | 1   | 1     | 0     | 0     | $\text{Not}(x) \text{ And } \text{Not}(y) \text{ And } \text{Not}(z)$ |
|   | 0   | 0   | 1   | 0   | 0     | 0     | 0     |                                                                       |
| → | 0   | 1   | 0   | 1   | 0     | 1     | 0     | $\text{Not}(x) \text{ And } y \text{ And } \text{Not}(z)$             |
|   | 0   | 1   | 1   | 0   | 0     | 0     | 0     |                                                                       |
| → | 1   | 0   | 0   | 1   | 0     | 0     | 1     | $x \text{ And } \text{Not}(y) \text{ And } \text{Not}(z)$             |
|   | 1   | 0   | 1   | 0   | 0     | 0     | 0     |                                                                       |
|   | 1   | 1   | 0   | 0   | 0     | 0     | 0     |                                                                       |
|   | 1   | 1   | 1   | 0   | 0     | 0     | 0     |                                                                       |

1. Focus on the rows where  $f = 1$
2. For each such row  $i$ , define a function  $f_i$  which equals 1 in row  $i$  and 0 elsewhere. Define  $f_i$  to be a conjunction of all of  $f$ 's variables or their negations, as the variable is 1 or 0 in the  $i$ 'th row

# Boolean function synthesis

|   | $x$ | $y$ | $z$ | $f$ | $f_1$ | $f_3$ | $f_5$ |                                   |
|---|-----|-----|-----|-----|-------|-------|-------|-----------------------------------|
| → | 0   | 0   | 0   | 1   | 1     | 0     | 0     | (Not(x) And Not(y) And Not(z)) Or |
|   | 0   | 0   | 1   | 0   | 0     | 0     | 0     |                                   |
| → | 0   | 1   | 0   | 1   | 0     | 1     | 0     | (Not(x) And y And Not(z)) Or      |
|   | 0   | 1   | 1   | 0   | 0     | 0     | 0     |                                   |
| → | 1   | 0   | 0   | 1   | 0     | 0     | 1     | (x And Not(y) And Not(z))         |
|   | 1   | 0   | 1   | 0   | 0     | 0     | 0     |                                   |
|   | 1   | 1   | 0   | 0   | 0     | 0     | 0     |                                   |
|   | 1   | 1   | 1   | 0   | 0     | 0     | 0     |                                   |

1. Focus on the rows where  $f = 1$
2. For each such row  $i$ , define a function  $f_i$  which equals 1 in row  $i$  and 0 elsewhere.  
Define  $f_i$  to be a conjunction of all of  $f$ 's variables or their negations, as the variable is 1 or 0 in the  $i$ 'th row
3. Define  $f$  to be the disjunction of all these conjunctions

# Boolean function synthesis

| $x$ | $y$ | $z$ | $f$ |
|-----|-----|-----|-----|
| 0   | 0   | 0   | 1   |
| 0   | 0   | 1   | 0   |
| 0   | 1   | 0   | 1   |
| 0   | 1   | 1   | 0   |
| 1   | 0   | 0   | 1   |
| 1   | 0   | 1   | 0   |
| 1   | 1   | 0   | 0   |
| 1   | 1   | 1   | 0   |

$$\begin{array}{l}
 (\text{Not}(x) \text{ And } \text{Not}(y) \text{ And } \text{Not}(z)) \text{ Or } \\
 (\text{Not}(x) \text{ And } y \text{ And } \text{Not}(z)) \text{ Or } \\
 (x \text{ And } \text{Not}(y) \text{ And } \text{Not}(z))
 \end{array}
 \left. \vphantom{\begin{array}{l} \\ \\ \end{array}} \right\} \begin{array}{l} \text{Disjunctive} \\ \text{Normal} \\ \text{Form} \\ \text{(DNF)} \end{array}$$

1. Focus on the rows where  $f = 1$
2. For each such row  $i$ , define a function  $f_i$  which equals 1 in row  $i$  and 0 elsewhere.  
Define  $f_i$  to be a conjunction of all of  $f$ 's variables or their negations, as the variable is 1 or 0 in the  $i$ 'th row
3. Define  $f$  to be the disjunction of all these conjunctions

# Boolean function synthesis

| $x$ | $y$ | $z$ | $f$ |
|-----|-----|-----|-----|
| 0   | 0   | 0   | 1   |
| 0   | 0   | 1   | 0   |
| 0   | 1   | 0   | 1   |
| 0   | 1   | 1   | 0   |
| 1   | 0   | 0   | 1   |
| 1   | 0   | 1   | 0   |
| 1   | 1   | 0   | 0   |
| 1   | 1   | 1   | 0   |

$$\begin{array}{l}
 (\text{Not}(x) \text{ And } \text{Not}(y) \text{ And } \text{Not}(z)) \text{ Or } \\
 (\text{Not}(x) \text{ And } y \text{ And } \text{Not}(z)) \text{ Or } \\
 (x \text{ And } \text{Not}(y) \text{ And } \text{Not}(z))
 \end{array}
 \left. \vphantom{\begin{array}{l} \\ \\ \end{array}} \right\} \begin{array}{l} \text{Disjunctive} \\ \text{Normal} \\ \text{Form} \\ \text{(DNF)} \end{array}$$


---


$$\text{Not}(z) \text{ And } (\text{Not}(x) \text{ Or } \text{Not}(y)) \quad \text{(following simplification)}$$

1. Focus on the rows where  $f = 1$
2. For each such row  $i$ , define a function  $f_i$  which equals 1 in row  $i$  and 0 elsewhere.  
Define  $f_i$  to be a conjunction of all of  $f$ 's variables or their negations, as the variable is 1 or 0 in the  $i$ 'th row
3. Define  $f$  to be the disjunction of all these conjunctions

# Boolean function synthesis

| $x$ | $y$ | $z$ | $f$ |
|-----|-----|-----|-----|
| 0   | 0   | 0   | 1   |
| 0   | 0   | 1   | 0   |
| 0   | 1   | 0   | 1   |
| 0   | 1   | 1   | 0   |
| 1   | 0   | 0   | 1   |
| 1   | 0   | 1   | 0   |
| 1   | 1   | 0   | 0   |
| 1   | 1   | 1   | 0   |

## Lemma

Given a boolean function expressed as a truth table, we can always synthesize a formula that expresses that function

Proof: Use the truth table to construct the DNF (which is a formula)

## Theorem

Any boolean function can be represented as a formula containing only the operators And, Or, Not

Proof: Construct the function's truth table, then use the truth table to construct the DNF (which, by definition, uses only And, Or, Not).

# The expressive power of Nand

| $x \text{ Nand } y$ |     |      |
|---------------------|-----|------|
| $x$                 | $y$ | Nand |
| 0                   | 0   | 1    |
| 0                   | 1   | 1    |
| 1                   | 0   | 1    |
| 1                   | 1   | 0    |

| $x \text{ And } y$ |     |     |
|--------------------|-----|-----|
| $x$                | $y$ | And |
| 0                  | 0   | 0   |
| 0                  | 1   | 0   |
| 1                  | 0   | 0   |
| 1                  | 1   | 1   |

| $x \text{ Or } y$ |     |    |
|-------------------|-----|----|
| $x$               | $y$ | Or |
| 0                 | 0   | 0  |
| 0                 | 1   | 1  |
| 1                 | 0   | 1  |
| 1                 | 1   | 1  |

| $\text{Not}(x)$ |     |
|-----------------|-----|
| $x$             | Not |
| 0               | 1   |
| 1               | 0   |

## Observations

- $\text{Not}(x) = x \text{ Nand } x$
- $x \text{ And } y = \text{Not}(x \text{ Nand } y)$
- $x \text{ Or } y = \text{Not}(\text{Not}(x) \text{ And } \text{Not}(y))$   
(De Morgan)

## In other words:

- Not can be realized using Nand
- And can be realized using Nand
- Or can be realized using Nand

Theorem: Any Boolean function can be realized using only Nand.

Proof : Any Boolean function can be expressed using Not, And, and Or (DNF).  
Combined with the above observations, we get the theorem.

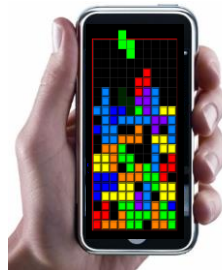
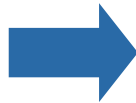
# The expressive power of Nand

Theorem: Any Boolean function can be realized using only Nand.

Conclusion: Any computer can be built from Nand gates only:

$x$  Nand  $y$

| $x$ | $y$ | Nand |
|-----|-----|------|
| 0   | 0   | 1    |
| 0   | 1   | 1    |
| 1   | 0   | 1    |
| 1   | 1   | 0    |



Computers:  
Machines that realize  
Boolean functions:  
 $f(\text{input bits}) = \text{output bits}$

OK, so we *can* build a computer from Nand gates only.

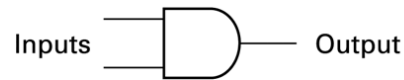
But how can we *actually* do it?

# Gates

- **Gate:** A device that computes a Boolean operation
  - Often implemented as (small) electronic circuits
  - Provide the building blocks from which computers are constructed
  - VLSI (Very Large Scale Integration) is a technology for placing millions of gates in a small chip

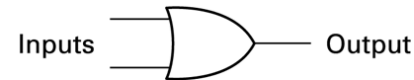
# A pictorial representation of AND, OR, XOR, and NOT gates as well as their input and output values

**AND**



| Inputs | Output |
|--------|--------|
| 0 0    | 0      |
| 0 1    | 0      |
| 1 0    | 0      |
| 1 1    | 1      |

**OR**



| Inputs | Output |
|--------|--------|
| 0 0    | 0      |
| 0 1    | 1      |
| 1 0    | 1      |
| 1 1    | 1      |

**XOR**



| Inputs | Output |
|--------|--------|
| 0 0    | 0      |
| 0 1    | 1      |
| 1 0    | 1      |
| 1 1    | 0      |

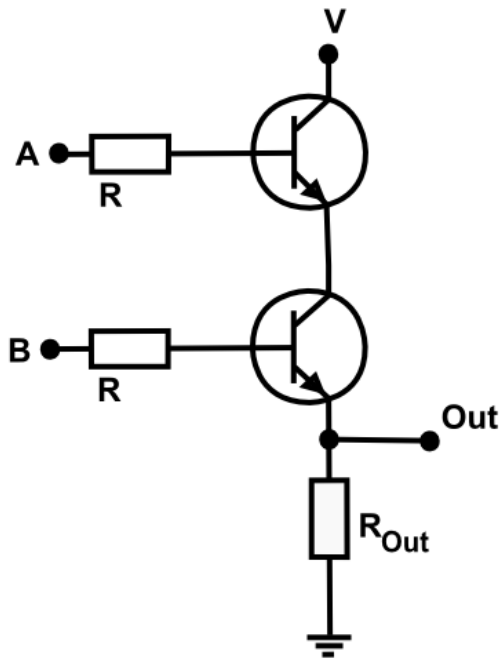
**NOT**



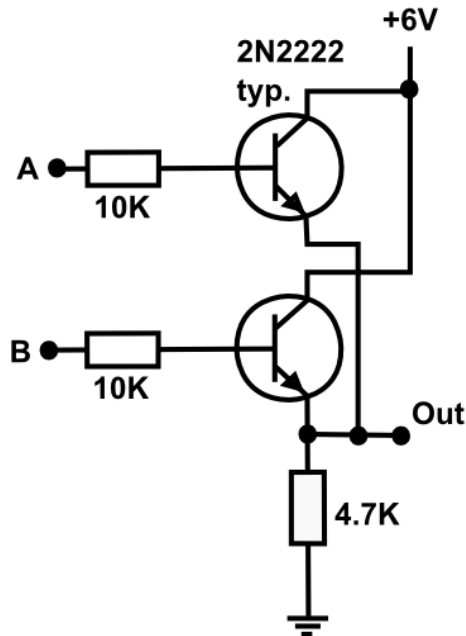
| Inputs | Output |
|--------|--------|
| 0      | 1      |
| 1      | 0      |

# Logical Gates using Transistors

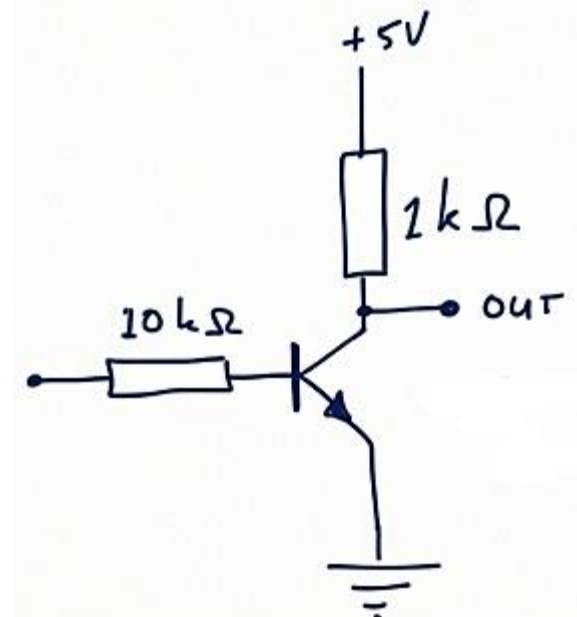
Transistor AND Gate



Transistor OR Gate

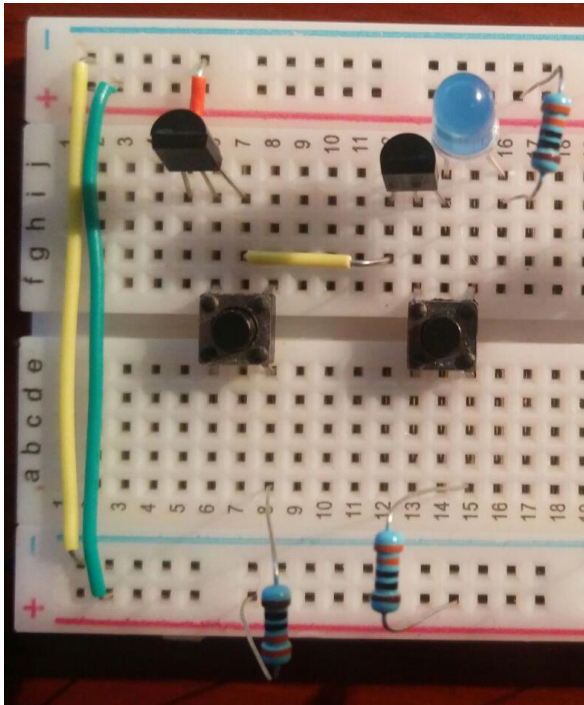


Transistor NOT Gate

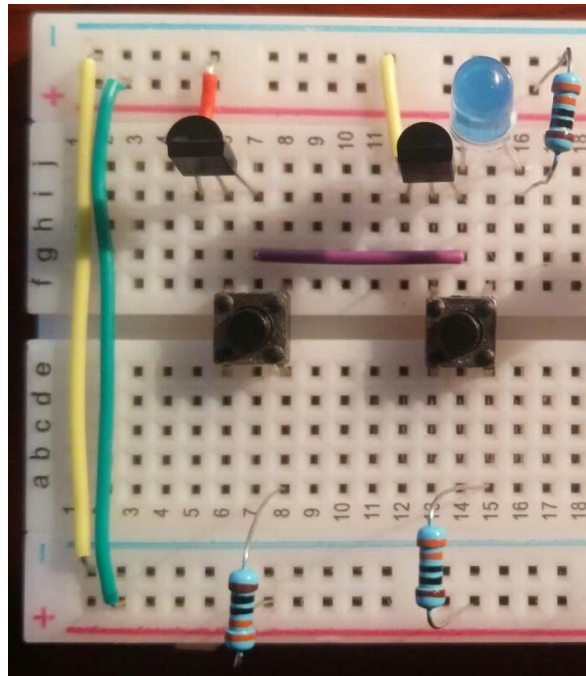


# Logical Gates using Transistors

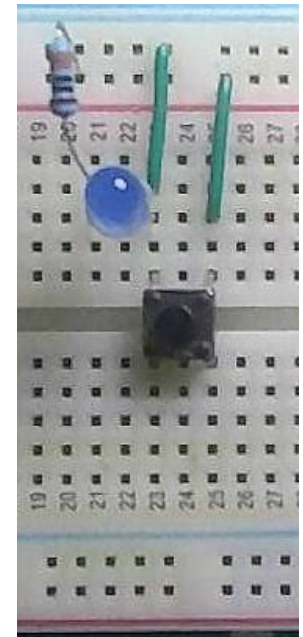
AND gate



OR gate

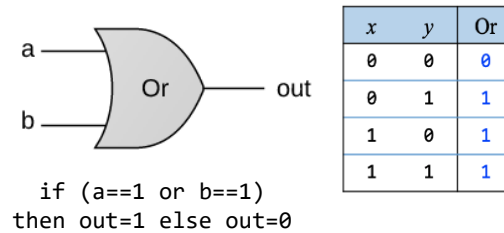
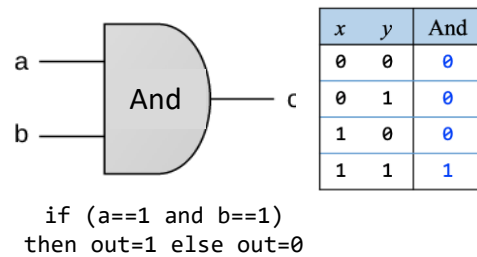


NOT gate

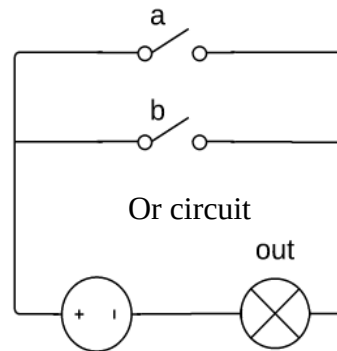
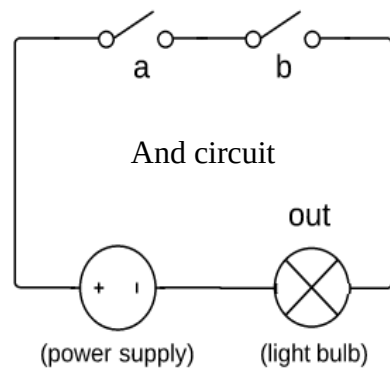


# Elementary gates

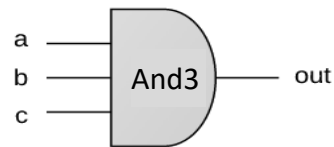
---



## Circuit implementations:

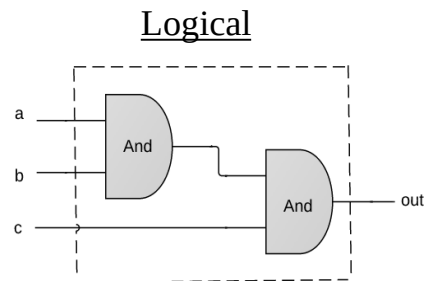
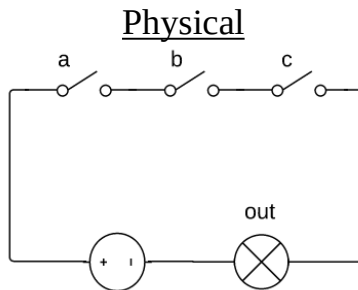


# Composite gates



```
if (a==1 and b==1 and c==1)
  then out=1 else out=0
```

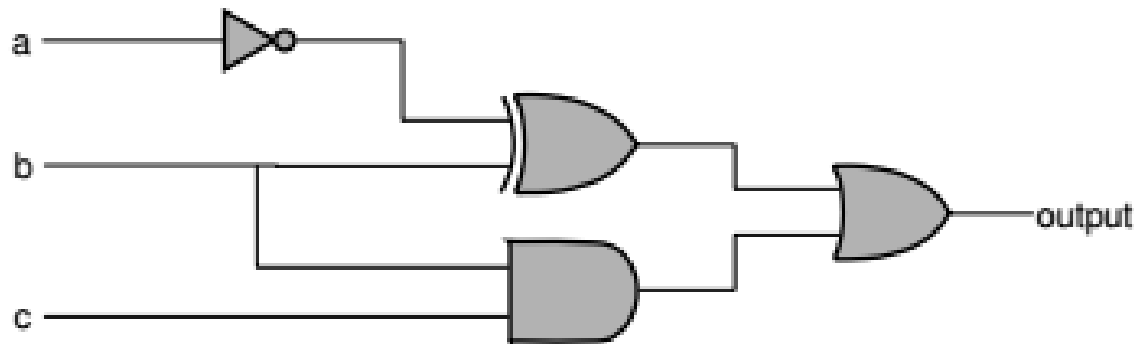
Possible implementations:



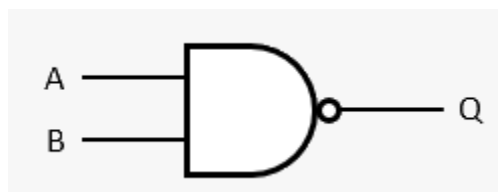
- This course does not deal with physical implementations (circuits, transistors, relays... that's EE, not CS)
- We focus only on logical implementations

# Exercise?

1. Can you write XOR with AND, OR and NOT?
2. What is truth table for below circuit?



3. Below is NAND Gate and its truth table. Can we use only NAND Gate to build NOT, OR and AND gates?

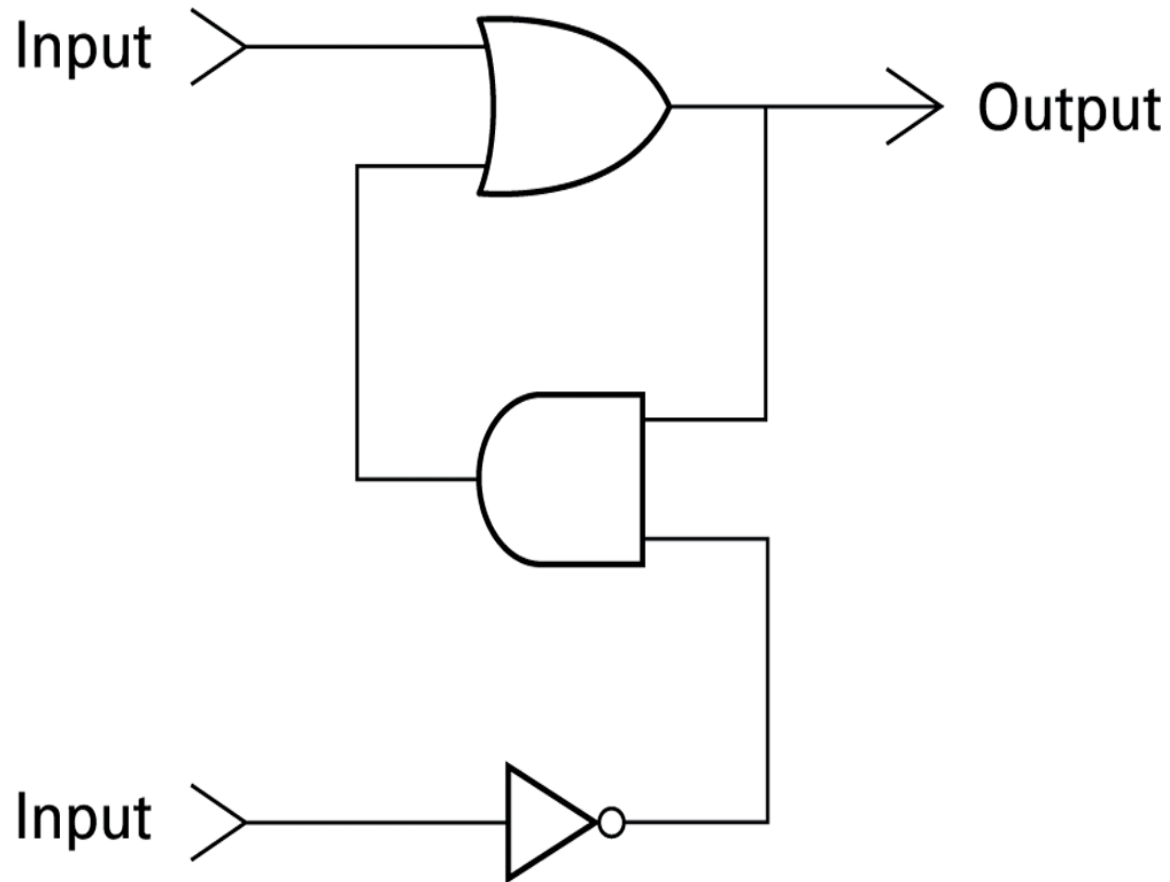


| A | B | Q |
|---|---|---|
| 1 | 1 | 0 |
| 1 | 0 | 1 |
| 0 | 1 | 1 |
| 0 | 0 | 1 |

# Flip-flops

- In addition to performing Boolean operations, gates can also be used to store information
- **Flip-flop:** A circuit built from gates that can store one bit.
  - One input line is used to set its stored value to 1
  - One input line is used to set its stored value to 0
  - While both input lines are 0, the most recently stored value is preserved

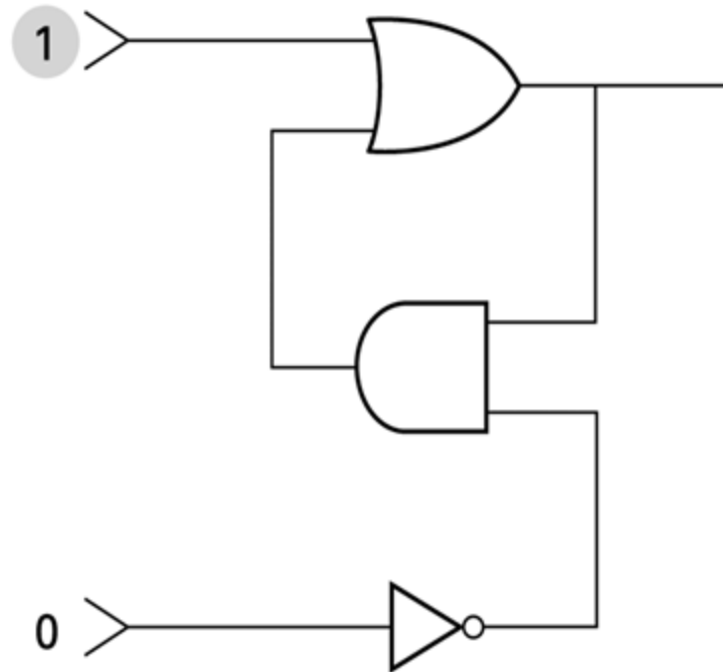
# A simple flip-flop circuit



# Setting the output of a flip-flop to 1

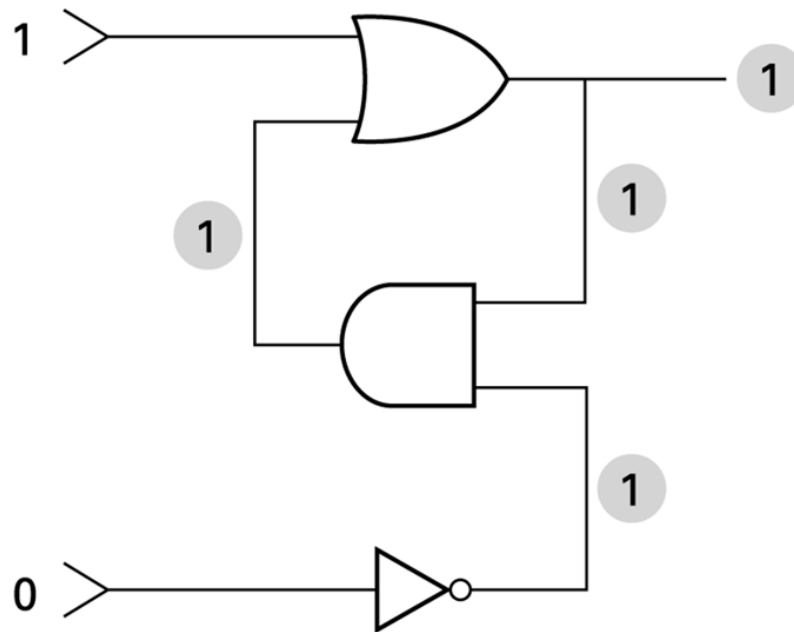
---

a. First, a 1 is placed on the upper input.



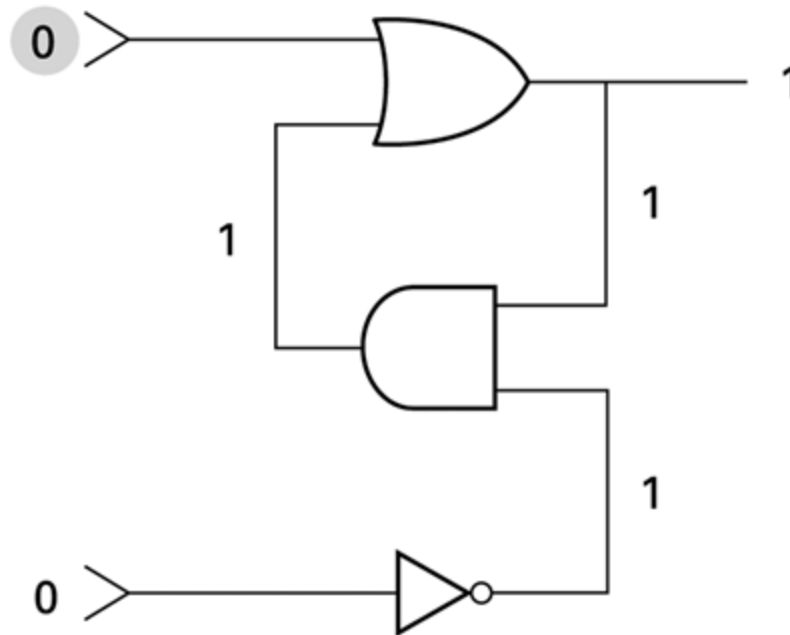
# Setting the output of a flip-flop to 1 (continued)

- 
- b.** This causes the output of the OR gate to be 1 and, in turn, the output of the AND gate to be 1.



# Setting the output of a flip-flop to 1 (continued)

- 
- c. Finally, the 1 from the AND gate keeps the OR gate from changing after the upper input returns to 0.



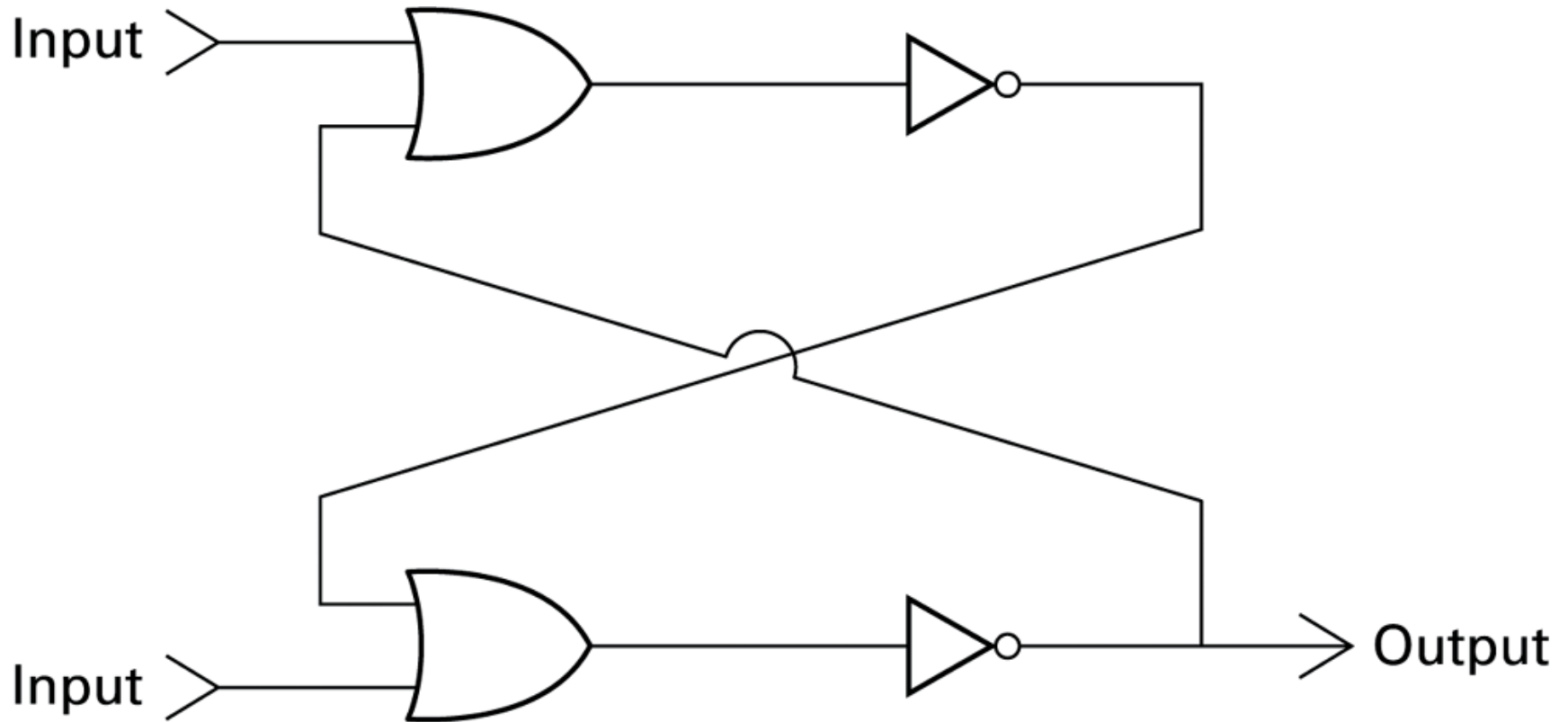
# Let's Build a Flip-Flop

<http://logic.ly/demo/>

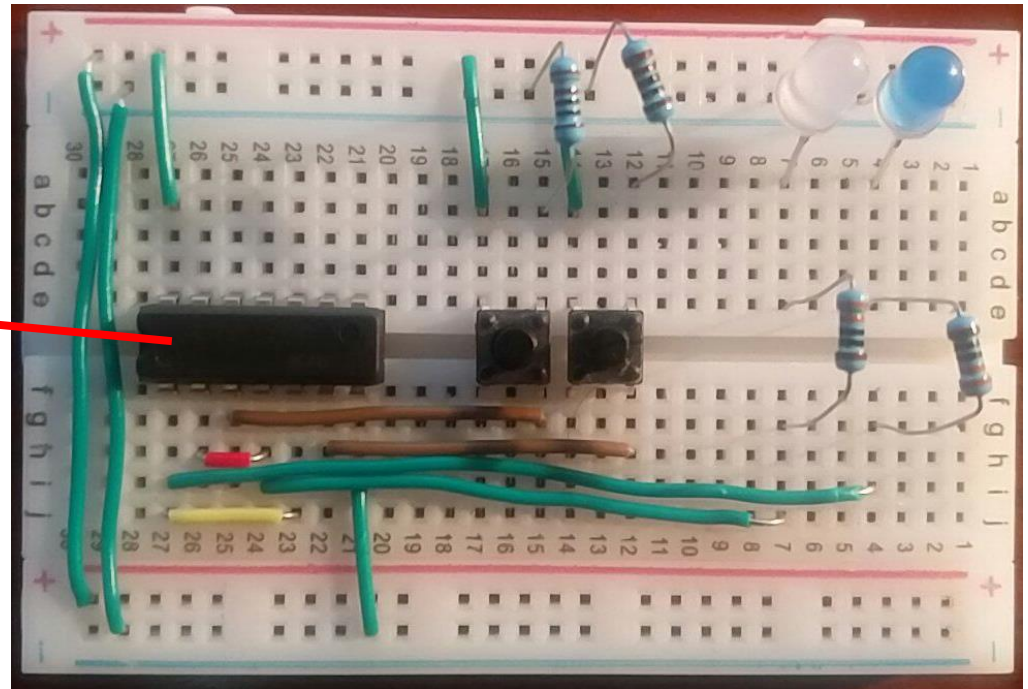
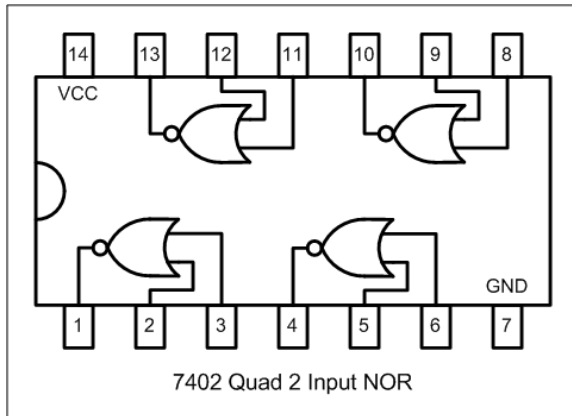
OR

Using [Logism Application](#)

# Another way of constructing a flip-flop



# Flip-flop implementation using 7402 integrated circuit (IC)



check other ICs here:

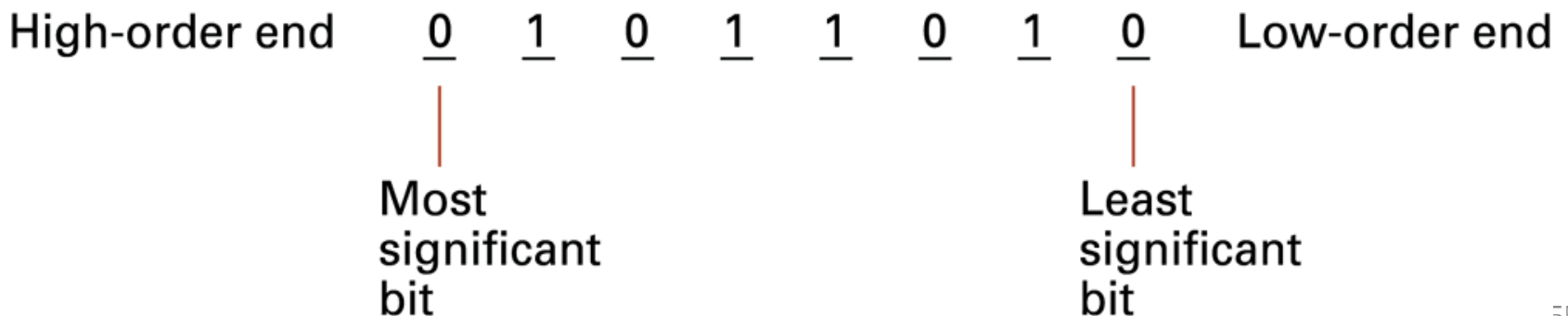
[https://en.wikipedia.org/wiki/List\\_of\\_7400\\_series\\_integrated\\_circuits](https://en.wikipedia.org/wiki/List_of_7400_series_integrated_circuits)

# Data Storage Hardware

- In addition to flip-flops, there are other fundamental bit-level data storage devices based on magnetic, optical or other technologies
- A data storage device can be volatile or non-volatile:
  - Volatile memory - holds its value until the power is turned off - For. ex: flip-flops
  - Non-volatile memory - holds its value after the power is off - For. ex: magnetic storage

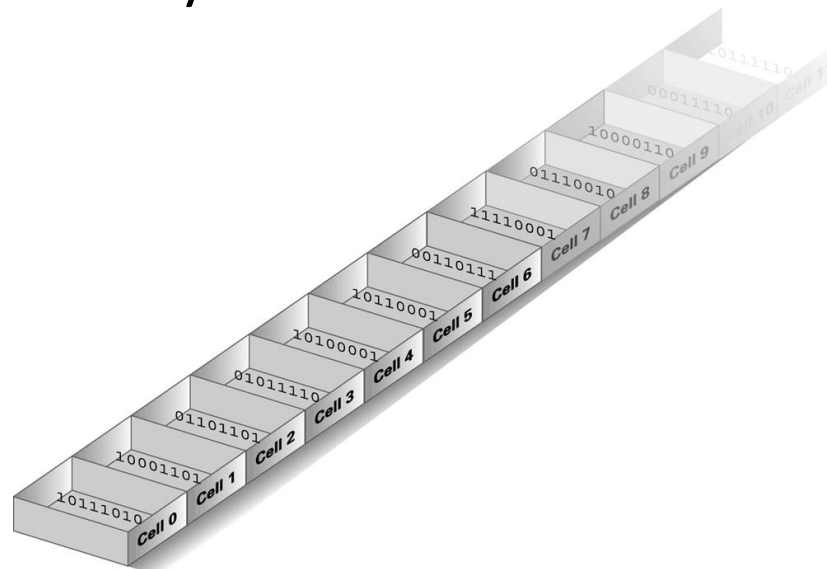
# Main Memory Cells

- **Cell:** A unit of main memory (typically 8 bits which is one **byte**)
  - **Most significant bit:** the bit at the left (high-order) end of the conceptual row of bits in a memory cell
  - **Least significant bit:** the bit at the right (low-order) end of the conceptual row of bits in a memory cell

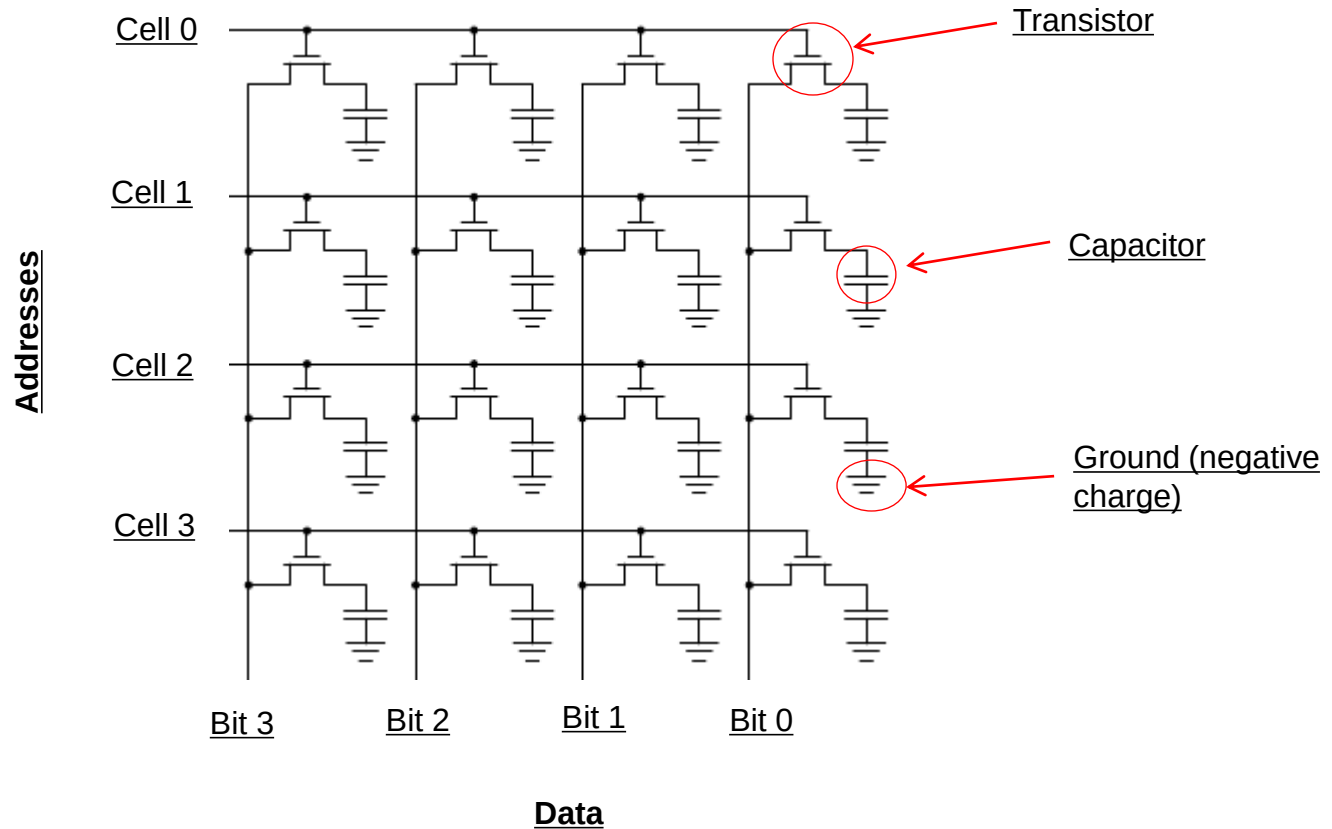


# Main Memory Addresses

- **Address:** A “name” that uniquely identifies one cell in the computer’s main memory
  - The names are actually numbers.
  - These numbers are assigned consecutively starting at zero.
  - Numbering the cells in this manner associates an order with the memory cells.



# Main Memory (DRAM)



# Memory Terminology

- **Random Access Memory (RAM):** Memory in which individual cells can be easily accessed in any order
  - **Dynamic RAM (DRAM):** each bit of data is stored in a separate capacitor within an integrated circuit. Because capacitors lose their charge quickly, they need to be refreshed with new charge every few milliseconds thus making the memory itself dynamic.
  - **Static RAM (SRAM):** Unlike DRAM, it can retain the stored values without need for periodical refreshment as long as there is power supply.
  - **Synchronous DRAM:** DRAM whose frequency of refreshments can be synchronized with CPU speed.

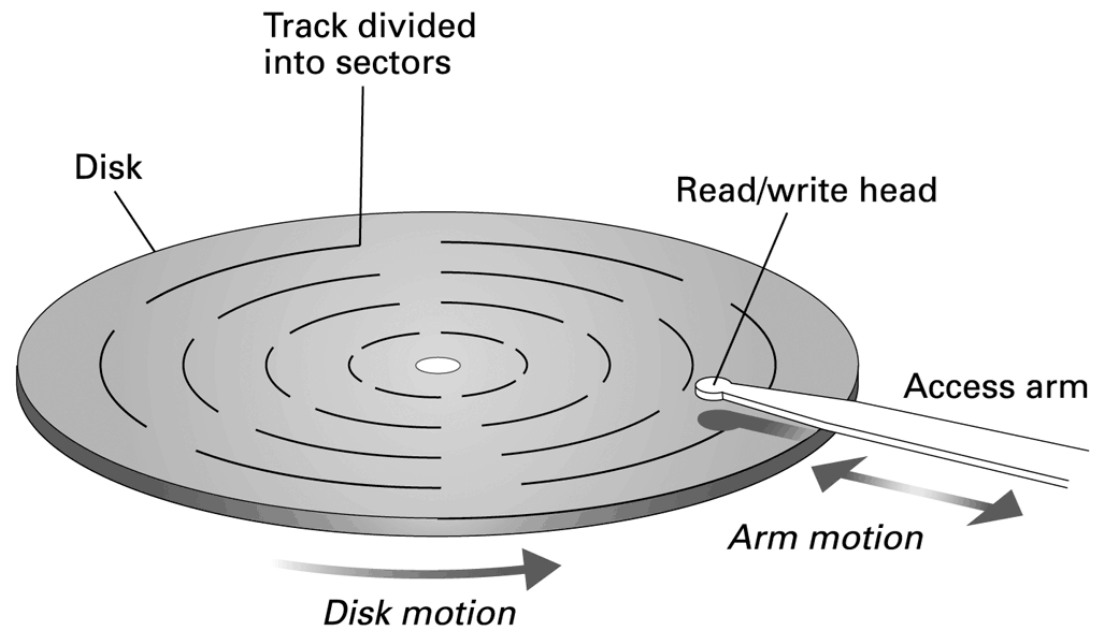
# Measuring Memory Capacity

- **Kilobyte:**  $2^{10}$  bytes = 1024 bytes
  - Example: 3 KB = 3 times 1024 bytes
- **Megabyte:**  $2^{20}$  bytes = 1,048,576 bytes
  - Example: 3 MB = 3 times 1,048,576 bytes
- **Gigabyte:**  $2^{30}$  bytes = 1,073,741,824 bytes
  - Example: 3 GB = 3 times 1,073,741,824 bytes

# Mass Storage

- Additional devices:
  - Magnetic disks
  - CDs
  - DVDs
  - Magnetic tape
  - Flash drives
  - Solid-state disks
- Advantages over main memory
  - Less volatility
  - Larger storage capacities
  - Low cost
  - In many cases can be removed

# A magnetic disk storage system

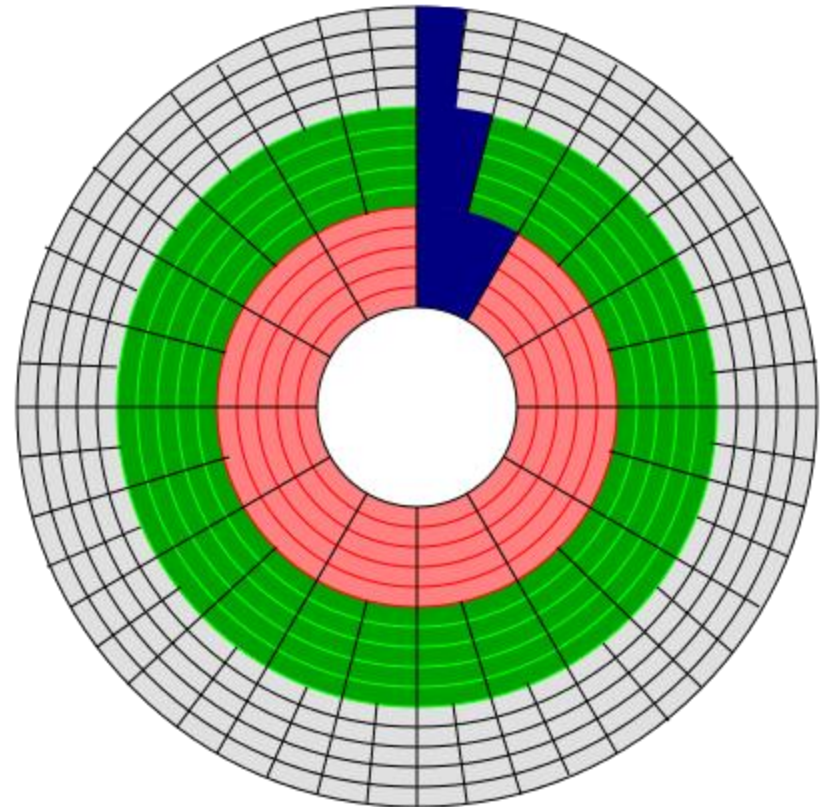


## ■ Characteristics:

- Seek time – the time to move the head to the right track
- Latency time – the time to rotate the disk for half a cycle
- Access time – seek time + latency time
- Transfer rate – the speed data transferred to or from the disk

# Zoned-bit Recording

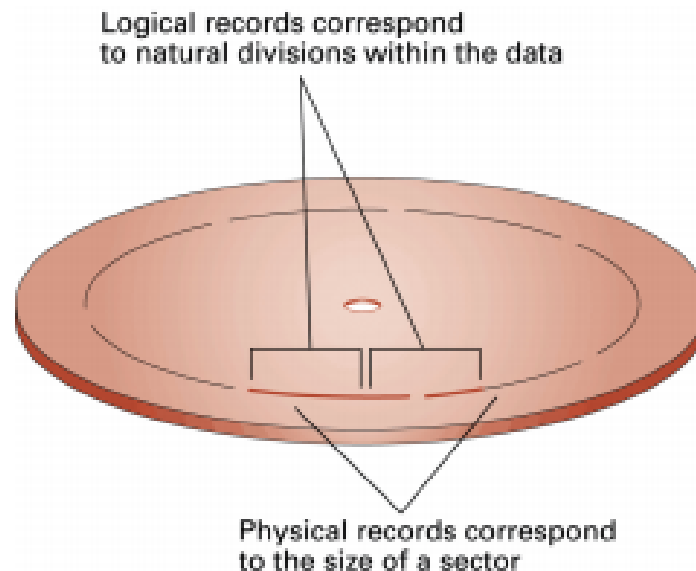
- Because outer tracks have larger space in sectors, they can be further split into smaller zoned-bits.
- This uses the space of the magnetic disc much efficiently and makes more sectors per track



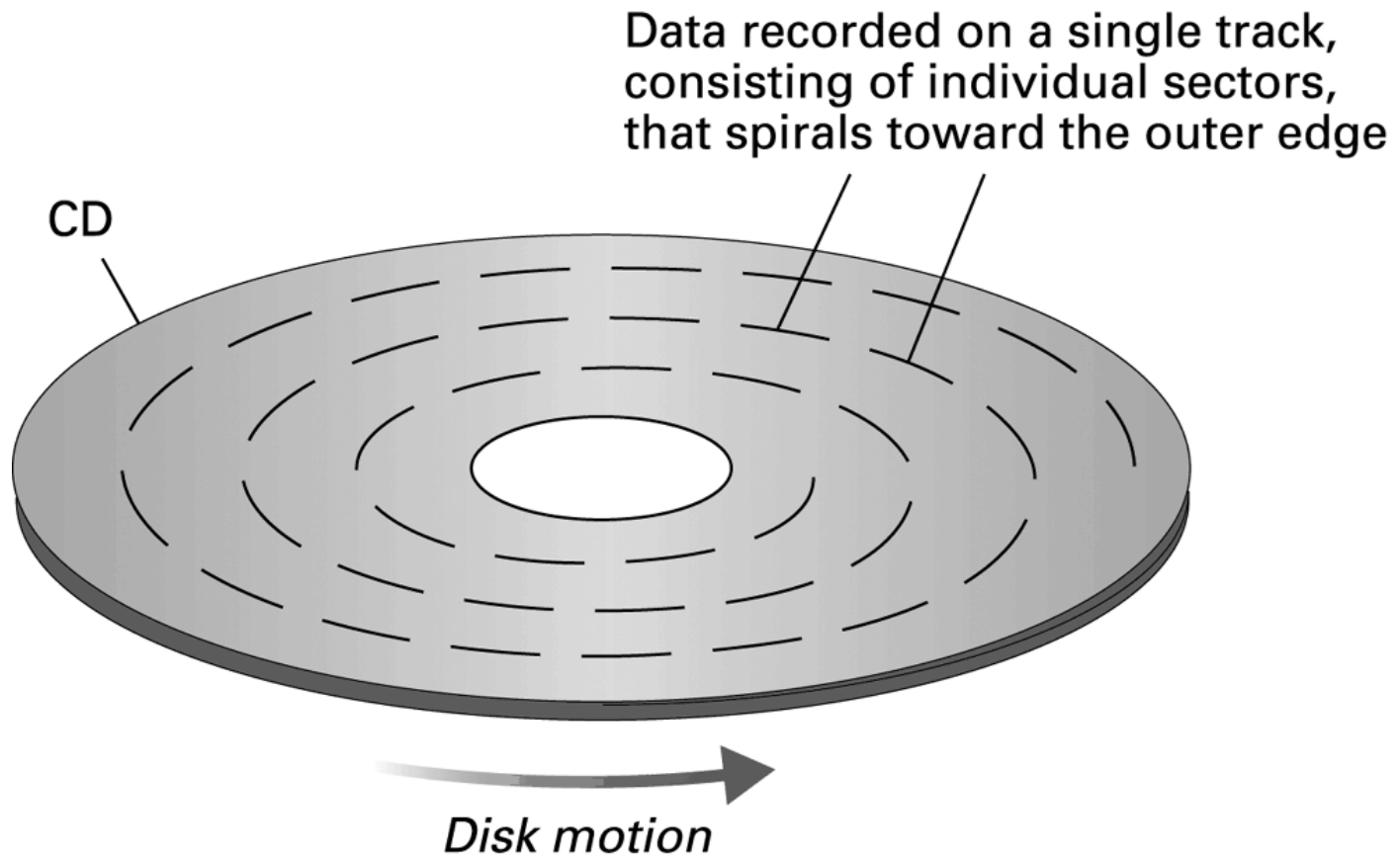
■ Sector 0

# Logical Record vs Physical Record

- ❑ Information stored in a mass storage is organized into files; each file is composed of many smaller units, called logical records
- ❑ The mapping between logical record and physical record is not one-to-one:



# CD storage



# Flash Drives

- **Flash Memory** – circuits that traps electrons in tiny silicon dioxide chambers
- Repeated erasing slowly damages the media
- Mass storage of choice for:
  - Digital cameras
  - Smartphones
- **SD Cards** provide GBs of storage



# Suggested Reading/Activity

- Tutorials:
  - Boolean Algebra: [https://www.electronics-tutorials.ws/boolean/bool\\_1.html](https://www.electronics-tutorials.ws/boolean/bool_1.html)
  - Logical Gates: [https://www.electronics-tutorials.ws/logic/logic\\_1.html](https://www.electronics-tutorials.ws/logic/logic_1.html)
  - Flip-flops: [https://www.electronics-tutorials.ws/sequential/seq\\_1.html](https://www.electronics-tutorials.ws/sequential/seq_1.html)
- How stuff works?
  - How DVDs work, <http://electronics.howstuffworks.com/dvd.htm>
  - How flash drives work, <http://computer.howstuffworks.com/flash-memory.htm>
  - How RAM works, <http://computer.howstuffworks.com/ram.htm>
  - How Analog-to-Digital (ADC) and Digital-to-Analog (DAC) sound conversion works: [http://www.msbtech.com/support/How\\_DACs\\_Work.php](http://www.msbtech.com/support/How_DACs_Work.php)
- Watch History:
  - Transistors, <https://youtu.be/OwS9aTE2Go4>